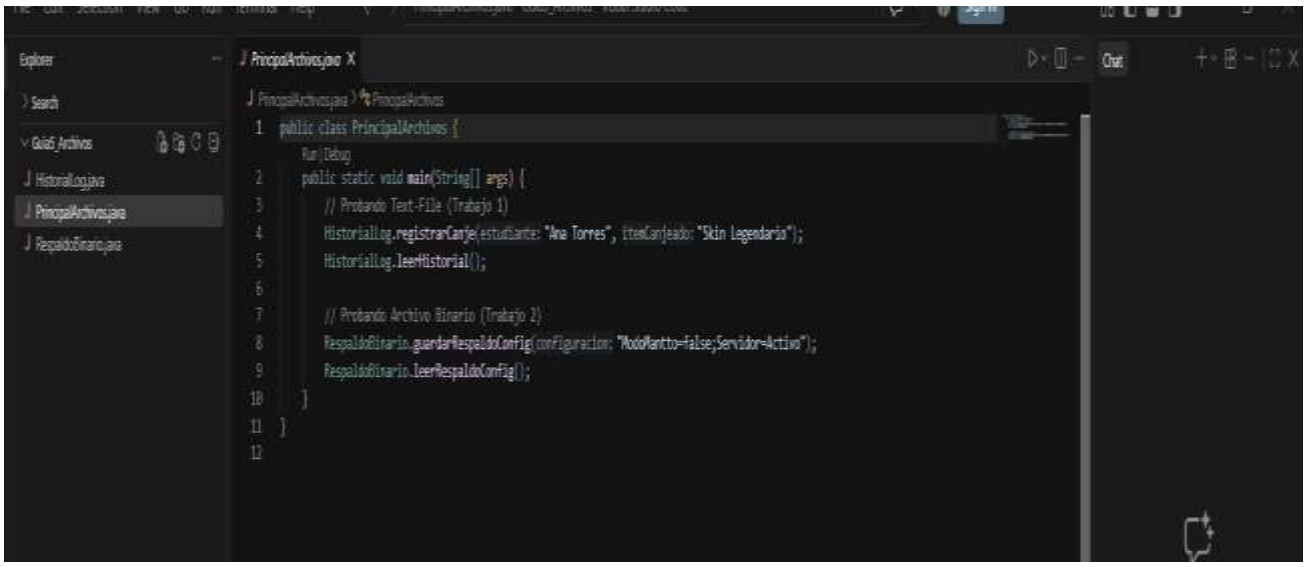
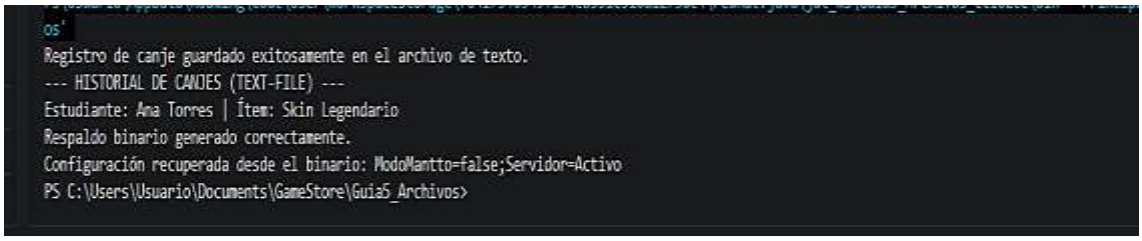
Manejo de Archivos de Texto (Text-File), Binarios y Excepciones en Java





¿Por qué es obligatorio usar bloques try-catch (o la cláusula throws) al manipular flujos de archivos en Java?

En Java, los métodos de la API java.io (FileWriter, PrintWriter, BufferedReader, FileReader, ObjectOutputStream, ObjectInputStream, etc.) están declarados con la cláusula throws IOException. Esto significa que cualquier código que los invoque puede fallar por causas externas al programa que no se pueden prever al compilar: el archivo puede no existir, el disco puede estar lleno, puede faltar permiso de escritura, el archivo puede estar bloqueado por otro proceso, o la ruta puede ser inválida.

Como IOException es una excepción verificada (checked exception, hija directa de Exception y no de RuntimeException), el compilador obliga al programador a reconocerla de una de dos formas: capturándola con un try-catch, o declarándola en la firma del método con throws IOException para que el llamador la maneje. Si ninguna de las dos se escribe, el programa no compila.

Usar try-catch (idealmente combinado con try-with-resources) no es solo una formalidad del compilador: garantiza que, si algo falla, el flujo se cierre correctamente, el programa muestre un mensaje amigable y siga funcionando en lugar de cerrarse de forma abrupta. En RespaldoBinario se atrapan además ClassNotFoundException e

IOException juntas, porque readObject() puede fallar por cualquiera de las dos razones al reconstruir un objeto guardado.